

M.SINDHUJA

2403A52060

BATCH 03

TASK 1: Select Word List

```
word_list = [
    'cat', 'dog', 'apple', 'banana', 'run', 'walk', 'happy', 'sad',
    'computer', 'science', 'mountain', 'ocean', 'city', 'country',
    'book', 'read', 'write', 'learn', 'teach', 'music', 'art',
    'love', 'hate', 'peace', 'war', 'time', 'space', 'earth', 'sky',
    'friend', 'enemy', 'car', 'bike', 'house', 'home', 'food', 'water',
    'sun', 'moon', 'star', 'tree', 'flower', 'bird', 'fish',
    'money', 'work', 'play', 'dream', 'reality', 'journey'
]
print(f"Word list created with {len(word_list)} words.")
print(word_list)
```

Word list created with 50 words.

['cat', 'dog', 'apple', 'banana', 'run', 'walk', 'happy', 'sad', 'computer', 'science', 'mountain', 'ocean', 'city', 'country', 'book', 'read', 'w

```
get_ipython().system('pip install spacy')
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.24.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.12.3)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (26.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy)
```

```
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spa
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.1.4)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (1.3.3)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (0.1.5)
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->spacy) (0.24.0)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (0.23.0)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (7.5.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0,>=0.4.2->spacy) (2.1.
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (8.3.
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (13.9
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spac
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->typer-slim<1.0.
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->typer-slim<1.
Requirement already satisfied: mdurl~>0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer>=0.24.0->typ
```

```
get_ipython().system('python -m spacy download en_core_web_md')
```

Collecting en-core-web-md==3.8.0

Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_md-3.8.0/en_core_web_md-3.8.0-py3-none-any.whl (33.5 MB)

33.5/33.5 MB 55.4 MB/s eta 0:00:00

Installing collected packages: en-core-web-md

Successfully installed en-core-web-md-3.8.0

✓ Download and installation successful

You can now load the package via `spacy.load('en_core_web_md')`

⚠ Restart to reload dependencies

If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

```
import spacy
```

```
nlp = spacy.load('en_core_web_md')
```

```
print("spaCy model 'en_core_web_md' loaded successfully.")
```

```
spaCy model 'en_core_web_md' loaded successfully.
```

```
word_embeddings = {}
```

```
for word in word_list:
```

```
    # Process the word to get a spaCy Doc object
```

```
    doc = nlp(word)
```

```
    # Check if the word has a vector (some words might not if they are out-of-vocabulary)
```

```
    if doc.has_vector:
```

```
        word_embeddings[word] = doc.vector
```

```
    else:
```

```

word_embeddings[word] = None # Or handle as appropriate
print(f"Extracted embeddings for {len(word_embeddings)} words.")
# Display a sample of words and their vectors
print("\nSample word embeddings (first 5 elements of vector):")
sample_count = 0
for word, vector in word_embeddings.items():
    if vector is not None:
        print(f"Word: {word}, Vector (sample): {vector[:5]}")
        sample_count += 1
    if sample_count >= 5:
        break

```

Extracted embeddings for 50 words.

```

Sample word embeddings (first 5 elements of vector):
Word: cat, Vector (sample): [-0.72483  0.42538  0.025489 -0.39807  0.037463]
Word: dog, Vector (sample): [-0.72483  0.42538  0.025489 -0.39807  0.037463]
Word: apple, Vector (sample): [-0.6334  0.18981 -0.53544 -0.52658 -0.30001]
Word: banana, Vector (sample): [-0.6334  0.18981 -0.53544 -0.52658 -0.30001]
Word: run, Vector (sample): [-0.73351  0.41392 -0.4425  -0.29127 -0.096179]

```

TASK 2: Apply t-SNE

```

from sklearn.manifold import TSNE
import numpy as np

# Filter out words that didn't have embeddings
filtered_words = [word for word, vector in word_embeddings.items() if vector is not None]
filtered_vectors = np.array([vector for word, vector in word_embeddings.items() if vector is not None])

# Initialize t-SNE model
# n_components=2 for 2-dimensional reduction
# random_state for reproducibility
tsne = TSNE(n_components=2, random_state=42, perplexity=10, n_iter=3000, learning_rate='auto')

# Apply t-SNE to the word vectors
reduced_vectors = tsne.fit_transform(filtered_vectors)

# Store the reduced vectors with their corresponding words
word_2d_coordinates = {}
for i, word in enumerate(filtered_words):
    word_2d_coordinates[word] = reduced_vectors[i]

print(f"Reduced {len(word_2d_coordinates)} word embeddings to 2 dimensions.")

# Display a sample of words and their 2D coordinates
print("\nSample word 2D coordinates:")
sample_count_2d = 0

```

```

for word, coords in word_2d_coordinates.items():
    print(f"Word: {word}, Coordinates: {coords}")
    sample_count_2d += 1
    if sample_count_2d >= 5:
        break

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to 'max_iter' in version 1.5 and will
warnings.warn(
Reduced 50 word embeddings to 2 dimensions.

```

Sample word 2D coordinates:

```

Word: cat, Coordinates: [330.3843 548.7233]
Word: dog, Coordinates: [213.85593 525.22375]
Word: apple, Coordinates: [-191.5535 880.9737]
Word: banana, Coordinates: [-125.54655 782.1554 ]
Word: run, Coordinates: [-109.35647 -353.94492]

```

```

x_coords = []
y_coords = []
words = []

```

```

for word, coords in word_2d_coordinates.items():
    x_coords.append(coords[0])
    y_coords.append(coords[1])
    words.append(word)

```

```

print(f"Extracted {len(words)} words and their 2D coordinates.")
print("Sample x_coords:", x_coords[:5])
print("Sample y_coords:", y_coords[:5])
print("Sample words:", words[:5])

```

Extracted 50 words and their 2D coordinates.

```

Sample x_coords: [np.float32(330.3843), np.float32(213.85593), np.float32(-191.5535), np.float32(-125.54655), np.float32(-109.35647)]
Sample y_coords: [np.float32(548.7233), np.float32(525.22375), np.float32(880.9737), np.float32(782.1554), np.float32(-353.94492)]
Sample words: ['cat', 'dog', 'apple', 'banana', 'run']

```

TASK 3 : Plot Visualization

```

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15, 12))
sns.scatterplot(x=x_coords, y=y_coords, s=50, alpha=0.8)

for i, word in enumerate(words):
    plt.annotate(word, (x_coords[i] + 5, y_coords[i] + 5), fontsize=9, alpha=0.7)

plt.title('2D t-SNE Projection of Word Embeddings', fontsize=16)

```

```
plt.xlabel('t-SNE Component 1', fontsize=12)
plt.ylabel('t-SNE Component 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



```
word_categories = {  
    'cat': 'animal',  
    'dog': 'animal',  
    'apple': 'fruit',  
    'banana': 'fruit',  
    'run': 'verb',  
    'walk': 'verb',  
    'happy': 'emotion',  
    'sad': 'emotion',  
    'computer': 'object/tech',  
    'science': 'abstract/concept',  
    'mountain': 'nature/geography',  
    'ocean': 'nature/geography',  
    'city': 'nature/geography',  
    'country': 'nature/geography',  
    'book': 'object/tech',  
    'read': 'verb',  
    'write': 'verb',  
    'learn': 'verb',  
    'teach': 'verb',  
    'music': 'object/tech',  
    'art': 'object/tech',  
    'love': 'verb/emotion',  
    'hate': 'verb/emotion',  
    'peace': 'emotion/concept',  
    'war': 'emotion/concept',  
    'time': 'abstract/concept',  
    'space': 'abstract/concept',  
    'earth': 'nature/geography',  
    'sky': 'nature/geography',  
    'friend': 'social/relation',  
    'enemy': 'social/relation',  
    'car': 'object/tech',  
    'bike': 'object/tech',  
    'house': 'object/tech',  
    'home': 'object/tech',  
    'food': 'object/tech',  
    'water': 'object/tech',  
    'sun': 'nature/geography',  
    'moon': 'nature/geography',  
    'star': 'nature/geography',  
    'tree': 'nature/geography',  
    'flower': 'nature/geography',  
    'bird': 'animal',  
    'fish': 'animal',  
    'money': 'object/concept',  
    'work': 'verb/concept',  
    'play': 'verb/concept',  
}
```

```
'dream': 'emotion/concept',
'reality': 'abstract/concept',
'journey': 'abstract/concept'
}

print(f"Assigned categories to {len(word_categories)} words.")
print("Sample word categories:", list(word_categories.items())[:5])
```

Assigned categories to 50 words.

Sample word categories: [('cat', 'animal'), ('dog', 'animal'), ('apple', 'fruit'), ('banana', 'fruit'), ('run', 'verb')]

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Create a DataFrame for easier plotting with seaborn
plot_data = pd.DataFrame({
    'x': x_coords,
    'y': y_coords,
    'word': words,
    'category': [word_categories[word] for word in words] # Assign category to each word
})

plt.figure(figsize=(18, 15))

# Use seaborn scatterplot with 'hue' for coloring by category
sns.scatterplot(
    data=plot_data,
    x='x',
    y='y',
    hue='category',
    s=100, # Increase point size for better visibility
    alpha=0.8,
    palette='tab20' # Use a palette with more distinct colors
)

# Annotate each point with its word
for i, row in plot_data.iterrows():
    plt.annotate(row['word'], (row['x'] + 5, row['y'] + 5), fontsize=9, alpha=0.7)

plt.title('2D t-SNE Projection of Word Embeddings by Category', fontsize=18)
plt.xlabel('t-SNE Component 1', fontsize=14)
plt.ylabel('t-SNE Component 2', fontsize=14)
plt.legend(title='Word Category', bbox_to_anchor=(1.05, 1), loc='upper left') # Place legend outside the plot
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout() # Adjust layout to prevent labels/legend overlapping
plt.show()
```


